

HALF ADDER

HDL Code • Code Description • Truth Table • Logic Diagram

Digital Logic Design Project

A Half Adder is a combinational logic circuit that adds two one-bit binary inputs and produces two outputs: Sum and Carry.

1. Introduction

A Half Adder adds two binary bits, A and B. Since it has no carry-in input, it is called a Half Adder.

Inputs: A, B

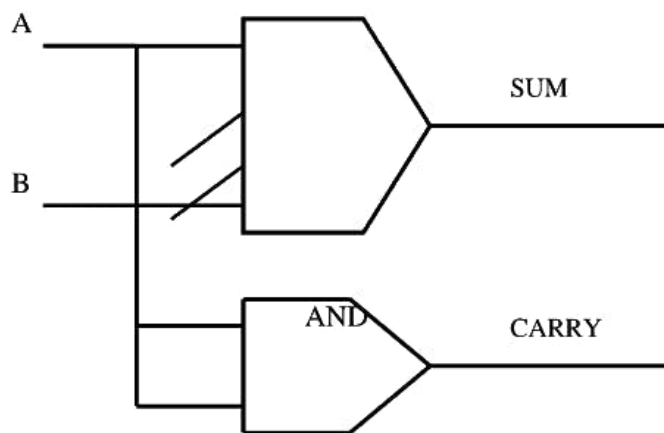
Outputs: Sum, Carry

The outputs are defined by the Boolean expressions:

Sum = A XOR B

Carry = A AND B

XOR HALF ADDER LOGIC DIAGRAM



2. Half Adder HDL Code

The following HDL code uses an XOR gate to generate Sum and an AND gate to generate Carry.

HalfAdder.hdl

```
CHIP HalfAdder {  
  IN a, b;  
  OUT sum, carry;  
  
  PARTS:  
    Xor(a=a, b=b, out=sum);  
    And(a=a, b=b, out=carry);  
}
```

3. Code Description

CHIP HalfAdder — declares the chip name.

IN a, b; — declares the two one-bit inputs.

OUT sum, carry; — declares the two outputs.

PARTS: — begins the internal component definitions.

Xor(a=a, b=b, out=sum); — produces the Sum output.

And(a=a, b=b, out=carry); — produces the Carry output.

4. Truth Table

All possible combinations of the two input bits are shown below.

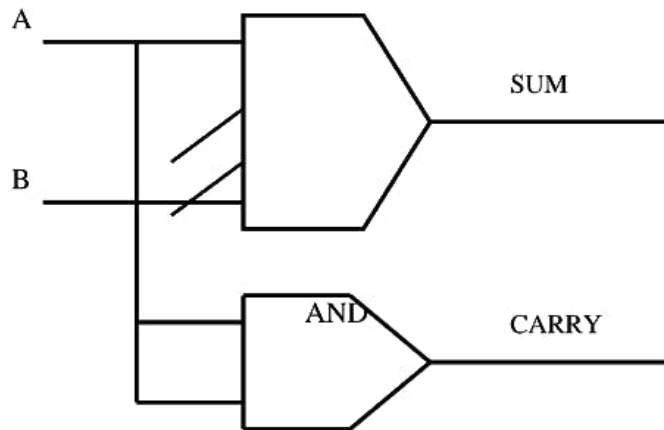
A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Observation: Sum is 1 when A and B are different. Carry is 1 only when both A and B are 1.

5. Logic Diagram

The Half Adder uses an XOR gate for Sum and an AND gate for Carry. Both gates receive the same inputs A and B.

XOR HALF ADDER LOGIC DIAGRAM



XOR: Sum output **AND:** Carry output

6. Working Examples

A=0, B=0: Sum=0, Carry=0

A=0, B=1: Sum=1, Carry=0

A=1, B=0: Sum=1, Carry=0

A=1, B=1: Sum=0, Carry=1, therefore $1 + 1 = 10$ ■.

7. Conclusion

The Half Adder is a basic combinational circuit for one-bit binary addition. Its Sum is generated by XOR and its Carry is generated by AND.

Final equations:

$$\text{Sum} = A \text{ XOR } B$$

$$\text{Carry} = A \text{ AND } B$$

END OF PROJECT DOCUMENT

FULL ADDER

HDL Code • Code Description • Truth Table • Logic Diagram

Digital Logic Design Project

A Full Adder is a combinational circuit that adds three one-bit inputs: A, B and Carry-In (C_{in}), producing Sum and Carry-Out (C_{out}).

1. Introduction

A Full Adder extends the Half Adder by including a Carry-In input. It is used as a building block for multi-bit binary addition.

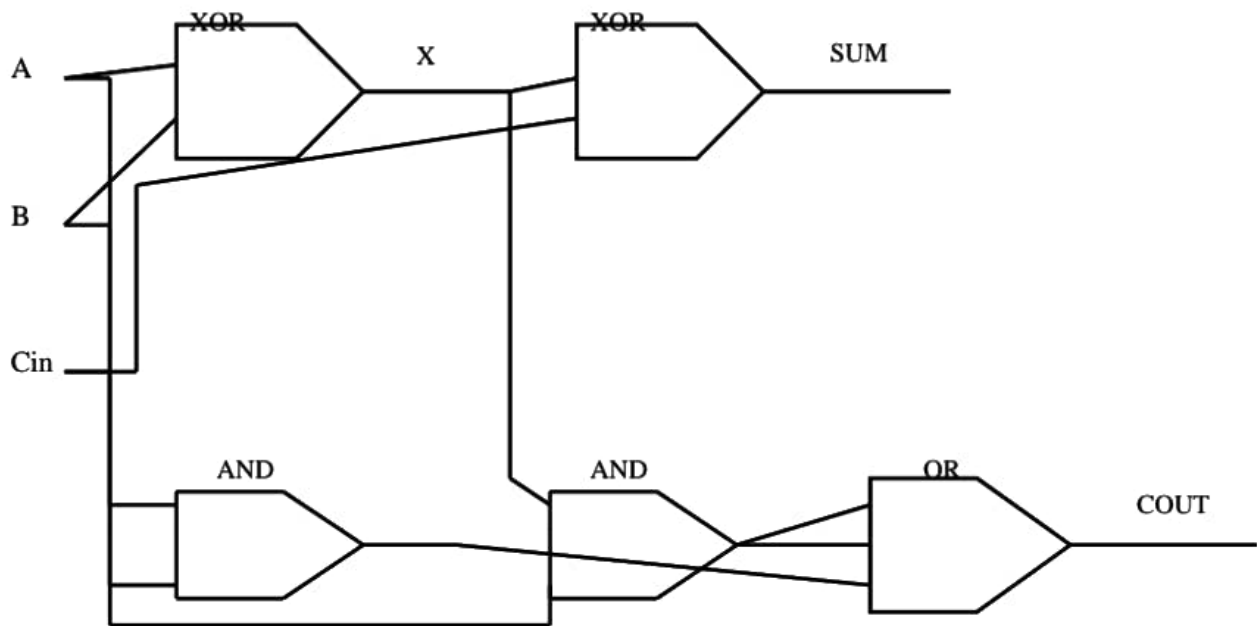
Inputs: A, B, Cin

Outputs: Sum, Cout

Boolean expressions:

$\text{Sum} = A \text{ XOR } B \text{ XOR } \text{Cin}$

$\text{Cout} = (A \text{ AND } B) \text{ OR } (\text{Cin AND } (A \text{ XOR } B))$



2. Full Adder HDL Code

This structural HDL uses two Half Adders and one OR gate.

FullAdder.hdl

```
CHIP FullAdder {  
    IN a, b, c;  
    OUT sum, carry;  
  
    PARTS:  
    HalfAdder(a=a, b=b, sum=abSum, carry=abCarry);  
    HalfAdder(a=abSum, b=c, sum=sum, carry=bcCarry);  
    Or(a=abCarry, b=bcCarry, out=carry);  
}
```

3. Code Description

CHIP FullAdder — declares the chip.

IN a, b, c; — declares three 1-bit inputs; c is Carry-In.

OUT sum, carry; — declares Sum and Carry-Out.

First HalfAdder — adds A and B, producing abSum and abCarry.

Second HalfAdder — adds abSum and c, producing the final Sum and bcCarry.

Or — combines abCarry and bcCarry to produce the final Carry-Out.



4. Truth Table

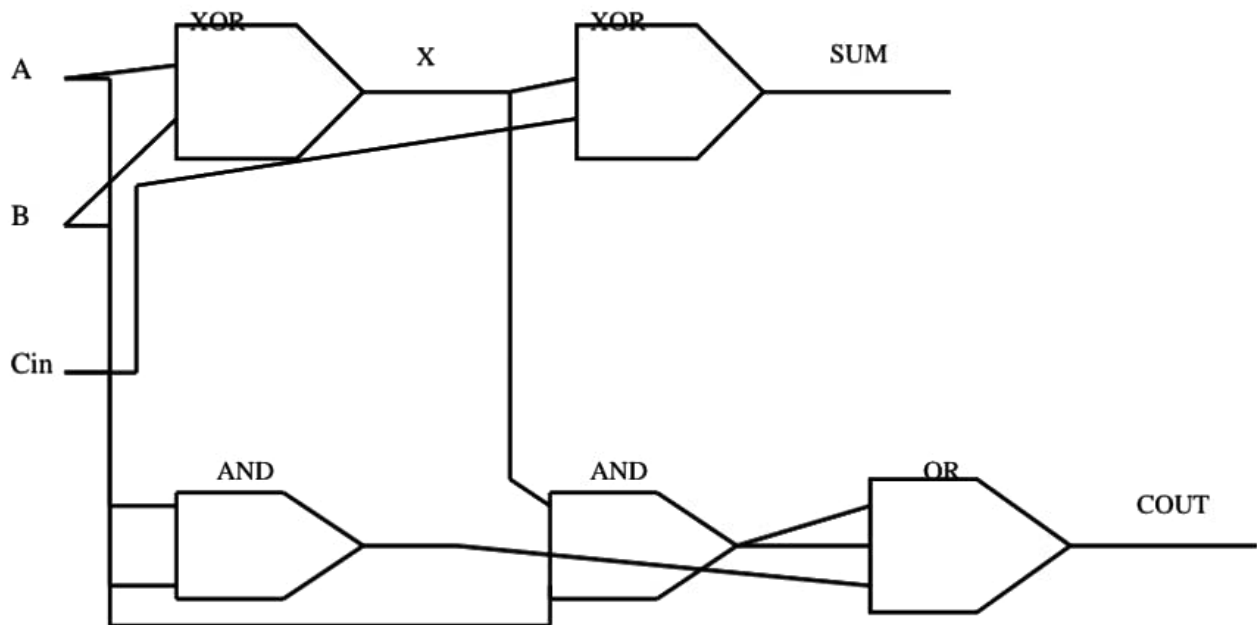
All eight possible input combinations are shown below.

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Observation: Sum is 1 for an odd number of 1 inputs. Cout is 1 when at least two inputs are 1.

5. Logic Diagram

A Full Adder can be constructed from two Half Adders and an OR gate. The first XOR/AND pair processes A and B; the second processes the intermediate Sum with Cin; the two carry signals are ORed.



6. Working Example

A=1, B=1, Cin=1

First stage: $1 + 1 \rightarrow$ intermediate Sum = 0, Carry = 1.

Second stage: $0 + 1 \rightarrow$ final Sum = 1, Carry = 0.

Carry-Out = 1 OR 0 = 1. Therefore the result is **11**■.

7. Conclusion

The Full Adder is a fundamental combinational circuit for binary arithmetic and is used to construct multi-bit adders such as Add16.

Key equations:

$$\text{Sum} = A \text{ XOR } B \text{ XOR } \text{Cin}$$

$$\text{Cout} = (A \text{ AND } B) \text{ OR } (\text{Cin} \text{ AND } (A \text{ XOR } B))$$

END OF PROJECT DOCUMENT

ADD16 GATE

HDL Code, Code Description, Functional Truth Table and Logic Diagram

1. Introduction

Add16 is a 16-bit binary adder. It accepts two 16-bit inputs and produces a 16-bit output containing their binary sum. The circuit is built as a chain of one Half Adder followed by fifteen Full Adders.

2. HDL Code

```
CHIP Add16 {
  IN a[16], b[16];
  OUT out[16];

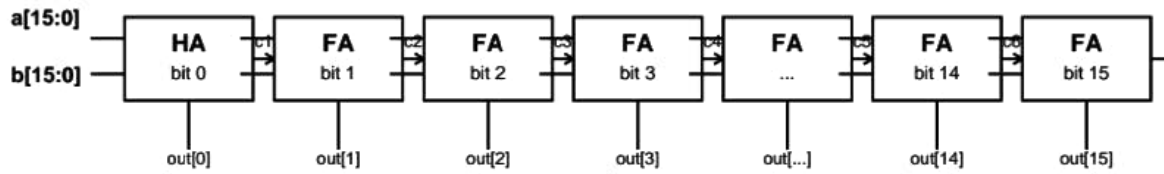
  PARTS:
    HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c1);
    FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
    FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
    FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
    FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
    FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c6);
    FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c7);
    FullAdder(a=a[7], b=b[7], c=c7, sum=out[7], carry=c8);
    FullAdder(a=a[8], b=b[8], c=c8, sum=out[8], carry=c9);
    FullAdder(a=a[9], b=b[9], c=c9, sum=out[9], carry=c10);
    FullAdder(a=a[10], b=b[10], c=c10, sum=out[10], carry=c11);
    FullAdder(a=a[11], b=b[11], c=c11, sum=out[11], carry=c12);
    FullAdder(a=a[12], b=b[12], c=c12, sum=out[12], carry=c13);
    FullAdder(a=a[13], b=b[13], c=c13, sum=out[13], carry=c14);
    FullAdder(a=a[14], b=b[14], c=c14, sum=out[14], carry=c15);
    FullAdder(a=a[15], b=b[15], c=c15, sum=out[15], carry=unused);
}
```

3. Code Description

HDL element	Description
a[16], b[16]	Two 16-bit input buses.
out[16]	16-bit output bus containing the sum.
HalfAdder	Adds the least significant bits, a[0] and b[0], with no incoming carry.
FullAdder	Each remaining bit is added together with the carry from the previous bit.
c1...c15	Internal carry signals connecting consecutive adder stages.
unused	The carry beyond bit 15 is discarded because the output is only 16 bits.

4. Add16 Logic Diagram

The carry output of each stage is connected to the carry input of the next higher bit. This creates a ripple-carry structure.



HA = Half Adder FA = Full Adder Carry propagates from bit 0 to bit 15.

5. Functional Truth Table

A complete truth table would contain 2^{32} input combinations, so representative functional cases are shown.

a (16-bit)	b (16-bit)	out (16-bit)	Operation
0000 0000 0000 0000	0000 0000 0000 0000	0000 0000 0000 0000	$0 + 0 = 0$
0000 0000 0000 0001	0000 0000 0000 0001	0000 0000 0000 0010	$1 + 1 = 2$
0000 0000 0000 0101	0000 0000 0000 0011	0000 0000 0000 1000	$5 + 3 = 8$
0000 0000 1111 1111	0000 0000 0000 0001	0000 0001 0000 0000	$255 + 1 = 256$
0000 0001 0000 0000	0000 0000 0000 0011	0000 0001 0000 0011	$256 + 3 = 259$
1111 1111 1111 1111	0000 0000 0000 0001	0000 0000 0000 0000	$65535 + 1 = 0^*$

* The mathematical result is 65,536, which requires 17 bits. Add16 keeps only the lower 16 bits, so the final carry is discarded.

6. Working Principle

The least significant bits are added first. Each Full Adder receives the carry generated by the previous stage. The carry therefore ripples toward the most significant bit.

7. Example

For $a = 0000\ 0000\ 0000\ 0101$ (5) and $b = 0000\ 0000\ 0000\ 0011$ (3), the output is $0000\ 0000\ 0000\ 1000$ (8).

8. Conclusion

The Add16 implementation demonstrates how smaller HalfAdder and FullAdder circuits can be combined to construct a 16-bit arithmetic unit.

ALU (ARITHMETIC LOGIC UNIT)

HDL Code, Code Description, Functional Table & Logic Diagram

1. Introduction

The ALU is a 16-bit arithmetic and logic unit used to perform arithmetic and logical operations on two 16-bit inputs. It can zero or negate each input, select between AND and addition, optionally negate the result, and generate zero and negative flags.

2. HDL Code

```
CHIP ALU {
  IN x[16], y[16],
  zx, nx, zy, ny, f, no;
  OUT out[16], zr, ng;

  PARTS:
    // Prepare x
    Mux16(a=x, b=false, sel=zx, out=x0);
    Not16(in=x0, out=xNot);
    Mux16(a=x0, b=xNot, sel=nx, out=x1);

    // Prepare y
    Mux16(a=y, b=false, sel=zy, out=y0);
    Not16(in=y0, out=yNot);
    Mux16(a=y0, b=yNot, sel=ny, out=y1);

    // AND or ADD
    And16(a=x1, b=y1, out=andOut);
    Add16(a=x1, b=y1, out=addOut);
    Mux16(a=andOut, b=addOut, sel=f, out=mainOut);

    // Optional output negation
    Not16(in=mainOut, out=negOut);
    Mux16(a=mainOut, b=negOut, sel=no, out=out);

    // Zero flag
    Or8Way(in=out[0..7], out=lo);
    Or8Way(in=out[8..15], out=hi);
    Or(a=lo, b=hi, out=anyOne);
    Not(in=anyOne, out=zr);

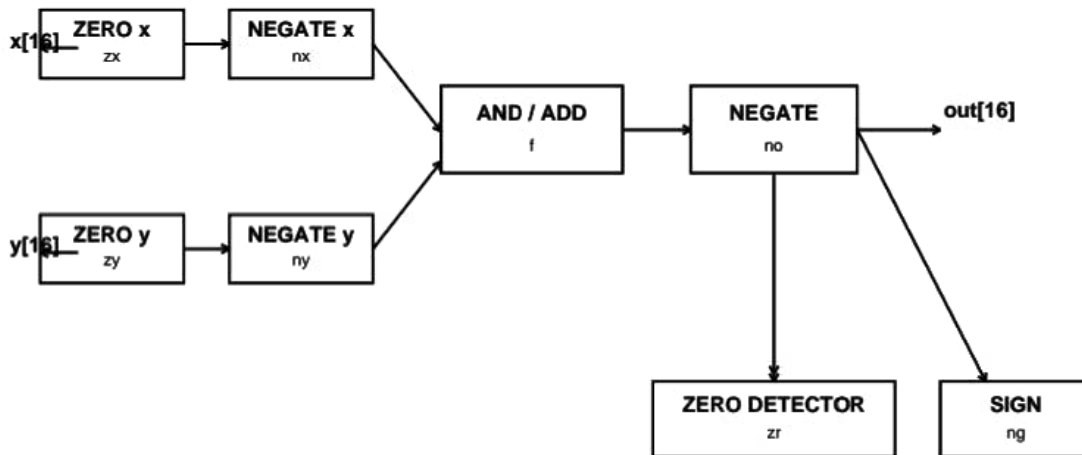
    // Negative flag
    And(a=out[15], b=true, out=ng);
}
```

3. Code Description

Signal / Part	Description
x[16], y[16]	Two 16-bit input buses.
zx, zy	When 1, the corresponding input is replaced by zero.
nx, ny	When 1, the corresponding input is bitwise negated.
f	Selects the main operation: 0 = x AND y, 1 = x + y.
no	When 1, the selected result is bitwise negated.
out[16]	The final 16-bit output.
zr	Zero flag; 1 when every output bit is 0.
ng	Negative flag; equal to out[15].

4. ALU Logic Diagram

The logic diagram shows the main data path. The x and y inputs are first conditioned by zero and negate controls. The ALU then selects AND or ADD using f, optionally negates the result using no, and generates zr and ng flags.



Control inputs: zx, nx, zy, ny, f, no

f = 0 → AND; f = 1 → ADD

no = 1 → invert selected result

zr = 1 when out = 0; ng = out[15]

5. ALU Functional Truth Table

A complete input truth table would contain 2^{32} input combinations for x and y . Therefore, the standard functional control table is used below. Each row specifies the control bits and the resulting function.

zx	nx	zy	ny	f	no	Output function
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	0	1	0	x
0	1	1	1	1	1	x
1	1	0	0	1	0	y
1	1	0	1	1	1	y
0	0	1	1	1	1	x + 1
0	1	1	1	1	0	x - 1
1	1	0	1	1	0	y - 1
1	1	0	0	1	1	-y
0	0	1	1	1	0	x - y
0	1	0	0	1	1	y - x
0	0	0	0	1	0	x + y
0	1	0	0	1	1	x + y
0	0	0	0	0	0	x & y
0	0	0	1	0	1	x y

6. Status Flags

zr: The zero flag is 1 when the 16-bit output is exactly zero. **ng:** The negative flag equals out[15], the most significant bit, which represents the sign in two's-complement notation.

7. Example

Example: For $x = 5$ and $y = 3$, select the control combination for $x + y$. The ALU produces out = 8, zr = 0, and ng = 0.

8. Conclusion

The ALU combines multiplexers, bitwise logic, arithmetic addition, negation, and status detection into one 16-bit unit. Its HDL implementation demonstrates how smaller digital components can be combined to create a complete processor arithmetic/logic block.